System Requirement Specifications (SRS)

Jerral Chua Fung Yong
Kataruka Mohika
Jethani Samarth
Sinha Armaan
Jain Sparsh
Faye Chan
Lai Yi

Team FUZE
College of Computing and Data Science, Nanyang Technological University

Submitted to—

Ajith Santhisenan

College of Computing and Data Science, Nanyang Technological University

Date: 14 September 2026
Revision 1.0

1 Table of Contents

1 Table of Contents 2
2 Problem Statement 5
3 Overview 5
3.1 Background 5
3.2 Overall Description 5
3.3 Definitions 7
4 Investigation & Analysis Methodology 7
4.1 System Investigation 7
4.2 Analysis Methodology 8
4.2.1 Feasibility study and requirements elicitation 8
4.2.2 System analysis and requirements specification 8
4.2.3 Scope and Limitations 8
4.2.4 Object-oriented design using Use Case and UML 9
4.2.5 Prototyping 16
4.2.6 Verification and Test Suites 16
5 Constraints 20
5.1 Distributor Rate Limits 20
5.2 Data Source Coverage 20
5.3 Missing Thermal Data 20
5.4 Missing Duty Cycle 20
5.5 Streaming and Scaling 20
5.6 Free-tier Hosting 20
5.7 Model Availability by Region 20
5.8 Project Schedule 21
5.9 Open Issues 21
Analysis volume. The analysis is meaningful only over a body of completed runs. A new account has none, and the interface presents an empty state rather than figures computed over one or two runs. Demonstration requires a seeded corpus. 22
6 Operational Requirements 23
6.1 Support 23
6.2 Application Services and Technical support 23
6.3 Administration Features 23
6.4 Availability and Failover 23
6.5 Audit Trail 23
7 Functional Requirements 23
7.1 Account and Session Management 23
7.2 Project Management 24
7.3 Board Design and Validation Workflow 24

7.4 The Constraint Rule Set 25
7.4.1 R1 voltage_overlap 25
7.4.2 R2 current_budget 25
7.4.3 R3 thermal_dissipation 26
7.4.4 R4 interface_role_match 26
7.4.5 R5 pin_budget 26
7.4.6 R6 availability 26
7.4.7 R7 temperature_rating 27
7.4.8 R8 footprint 27
7.4.9 R9 energy_budget 27
7.4.10 R10 rail_coverage 27
7.5 Bill of Materials and Reference Tools 27
7.6 Memory 28
7.7 Engineer Identity 28
7.8 The Brief 28
7.9 Bill of Materials Upload 29
7.10 Datasheet Upload 29
7.11 Component Data 29
7.12 Validation Analytics 29
8 Process Requirements 29
8.1 Interface Surface 29
8.2 The Event Stream 31
8.3 Determinism 32
8.4 Handling of Missing Data (DS-4) 32
8.5 Latency (DS-5) 33
8.6 Streaming Responsiveness (DS-6) 33
8.7 Bounded Repair (DS-7) 34
8.8 Sourceability (DS-9) 34
8.9 Model Fencing (DS-10) 34
8.10 Data integrity 35
8.11 Security 35
8.12 Analytics Integrity 35
9 Output Requirements 35
9.1 Verdicts 35
9.2 The Validation Trace and the Component Graph 35
9.3 The Bill of Materials 36
9.4 Export 36
9.5 Escalations 36
9.6 Analysis 36
10 Hardware Requirements 36
10.1 Network 36

10.2 Client Computers 36
10.3 Server 36
10.4 Bench Equipment 37
11 Software Requirements 37
11.1 Client 37
11.2 Frontend 37
11.3 Application Server 37
11.4 Orchestration 37
11.5 Constraint Engine 37
11.6 Data Sources 37
11.7 Language Model 37
11.8 Persistence 38
11.9 Testing 38
11.10 Licenses 38
12 Deployment Requirements 38

2 Problem Statement

A printed circuit board is built from a set of components that must work together. Someone has to choose those components before anything is drawn, and that choice becomes the bill of materials. Procurement buys from it and assembly builds from it, so an error in it survives every check that follows.

A bill of materials can be wrong in ways that no drawing tool detects. A 3.3-volt sensor placed on a 5-volt rail is destroyed on first power-up, and the schematic tool draws the connection without objection. A regulator rated for 600 mA feeding a radio that draws 500 mA and a display that draws 200 mA browns out under load. A linear regulator converting 5 V to 3.3 V at 400 mA turns 0.68 W into heat, which in a small package pushes the junction well past its rating. A part can be correct in every electrical respect and still be unavailable in the quantity a build needs.

None of these checks are difficult on their own. They are spread across dozens of datasheets, and no existing tool performs them for the board as a whole. Distributor search finds one part at a time. Computer-aided design tools check the drawing rather than the datasheet. Compatibility is a property of the whole board, and every available interface takes a single part as its unit of query.

A system is needed that reads a plain-language description of a board, fills it with real orderable parts, checks every rule against every part after every placement, and shows the engineer the field, the value and the source behind each judgement.

3 Overview

3.1 Background

Hardware engineers spend a large share of their working time confirming that parts chosen independently will work together. A survey of 439 engineers found that 77 percent spend five hours or more each week on this work. The average printed circuit board requires 2.9 respins before release, each costing about $28,500 and sixteen days. The cost of finding a problem late is high, and the cost of preventing it is low. That is the case for automating the checks.

The effort is also repeated. Most design projects make six or more component changes, and most boards lose at least one part to a supply, lifecycle or compliance problem. Every change invalidates the cross-checks already done on the parts around it, so the work is not done once but many times over the life of a design.

The sponsor has asked for a tool that answers one question. Given this set of parts and this application, does the board work, and if not, which number is wrong and where did it come from. Three conditions were stated. Judgements must be checkable against a named field in a named source. Supply must be treated as a design constraint, not an afterthought. The tool must be usable by someone who does not already know which cross-checks matter, because much of the wasted effort falls on junior engineers and student teams.

3.2 Overall Description

FUZE is a web application. An engineer signs in, creates a project, and describes a board in plain language. The system converts that description into component slots and power rails, fills each slot with

a real part from distributor data, and validates the board as a graph. Progress is streamed back to the browser as the run proceeds, so the engineer watches the board being built rather than waiting for a finished answer.

The system is split into two halves that are deliberately kept apart. A deterministic constraint engine decides what is broken. A language model decides what to do about it. The engine then re-checks the model's work. The engine holds no network client, no model client and no read of the system clock, so the same board produces the same verdicts on every run. No pass or fail verdict is ever produced by a language model.

When a rule fails, the engine computes the set of changes that are legal for that conflict and ranks them so the cheapest change comes first. The model selects one change from that set. The selection is checked for membership in the set before it is applied, and an illegal or late choice falls back to the least disruptive candidate. A slot is repaired at most three times, after which the conflict is raised to the engineer as an explicit trade-off.

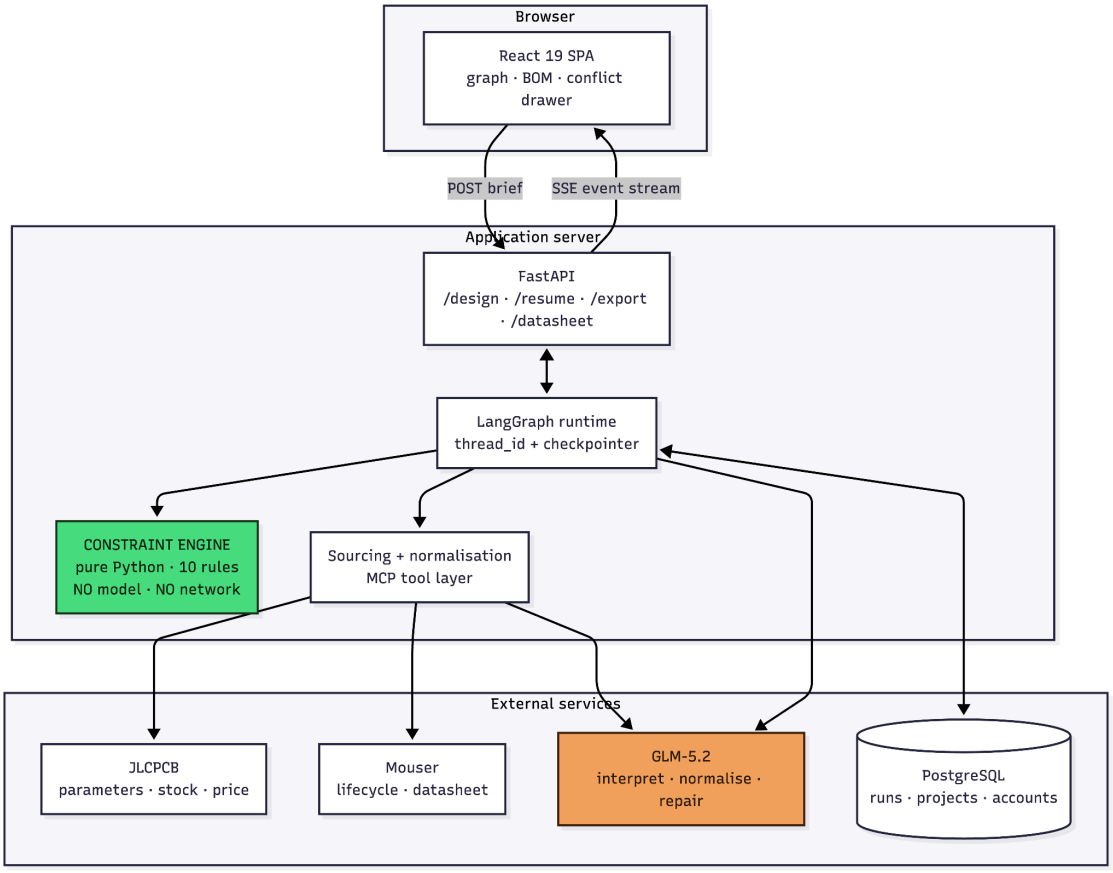


Figure 1: Overall system architecture, showing the deterministic constraint engine separated from the model and the external services.

The system does not draw a schematic, lay out or route a board, assign pins, produce a netlist, simulate a circuit, or emit manufacturing files. It validates a board at block level and produces a bill of materials that can be ordered.

3.3 Definitions

- Evaluation: One application of one rule to one part or rail within one run.
- Unchecked count: The number of evaluations of a given rule that could not be completed because a value the rule needed was not available.
- Conflict signature: The key formed from the rule that failed and the banded values that caused it, used to group comparable conflicts across runs.

4 Investigation & Analysis Methodology

4.1 System Investigation

The work an engineer does today was examined directly. Fifteen structured interviews were held across university electrical engineering programmes, a makerspace, an IEEE student branch, and competitive robotics and rocketry teams. The interviews changed the framing of the project. Engineers described the hard part not as picking a first set of parts, but as finding a replacement after a chosen part proves unavailable, one that still fits everything already committed around it.

The tools in use were also examined. Octopart, DigiKey, Mouser and the JLCPCB parts library expose distributor inventory as structured data, including price, stock and in some cases lifecycle status. Their unit of query is a single part, and none offers a way to ask a question about a set of parts. KiCad and Altium Designer check the drawn netlist for connectivity, pin type and unconnected nets. They do not read datasheets. Altium’s ActiveBOM and the Siemens supply-chain products add lifecycle and stock awareness to the bill of materials but do not validate parts against each other electrically.

Automated board synthesis was examined and ruled out. Tools in that space generate circuits or layouts. The interviews reproduced a finding already present in that literature: engineers want compatibility checking so that they can audit more than they want a generated board they cannot. Synthesis also needs pin-level connectivity, which is a much larger data problem than block-level validation and cannot be delivered responsibly in fourteen weeks.

The use of language models over engineering documents was examined last. A general assistant will state whether a sensor is compatible with a microcontroller, and will do so fluently whether it is right or wrong. An incorrect compatibility claim reads exactly like a correct one. The response adopted here is to give the model the judgement work it does well and to leave it structurally unable to produce a numbe

4.2 Analysis Methodology

4.2.1 Feasibility study and requirements elicitation

Requirements were elicited from the sponsor and from practising engineers. A preliminary round of fifteen interviews was completed and its findings were used to set the scope. A freedom-to-operate review covering USPTO full-text, Google Patents, Espacenet and the Intellectual Property Office of Singapore register was scheduled for the second week of the project, across the classification classes for computer-aided design of electronic circuits, design verification and rule checking, inventory and supply-chain management, and electronic catalogue and product-selection systems. The team’s preliminary assessment is that the contribution is a system architecture rather than an algorithm, and that the constraint checks themselves are textbook electrical engineering.

4.2.2 System analysis and requirements specification

The problem was analysed using object-oriented techniques. The main objects are the board, the slot, the part specification, the power rail, the rule, the verdict and the run. A part specification holds a typed value for each field together with the field name and the source it was read from, so that origin survives from the distributor payload through normalisation and into the engine. The board is held as a graph rather than a list, because the checks are relations between parts and rails rather than properties of parts alone.

Four design issues were identified as critical before implementation could begin.

- Origin must be carried through the whole architecture. It must therefore be part of the part specification type before any rule is written.
- Determinism must survive the use of a language model during normalisation of free-text catalogue values.
- The event contract must be defined early, because seven people cannot build different halves of a streaming system without an agreed event schema.
- Missing data must be handled explicitly by the engine rather than filled with an assumed value.

4.2.3 Scope and Limitations

The analysis covers business analysis, requirement analysis, data analysis, process analysis and application architecture. Business analysis states the rules the system enforces, the external interfaces it depends on, and the sponsor relationship. Requirement analysis defines the system inputs and outputs, the user requirements, and the functional and security requirements. Data analysis covers how component data is collected, validated, stored, normalised and retrieved with its origin intact. Process analysis covers the flow of a run from brief to bill of materials, and the points at which the run pauses for human input. Application architecture covers the interface surface, the event stream, the screens and the usability requirement that a judgement can be checked by someone who did not produce it.

The system validates at block level only. Its results depend on the quality and completeness of the parameters available to it. A component with a sparse catalogue entry will accumulate unchecked verdicts. That is the intended behaviour, and an unchecked result must never be read as validated.

4.2.4 Object-oriented design using Use Case and UML

The system was modelled with a use case diagram and a set of use case descriptions. One actor is human, the Engineer. Three are external systems: the Model Service, which selects a repair strategy from the set the engine computes; the Distributor Catalogue, which supplies component parameters, stock and price; and the Lifecycle Data Source, which supplies lifecycle status and datasheet links. The constraint engine and the workflow orchestrator are internal to the system and are described in the software requirements rather than modelled as actors. The use cases are grouped into account and session management, project management, the board design and validation workflow, the bill of materials and reference tools, and memory and analytics.

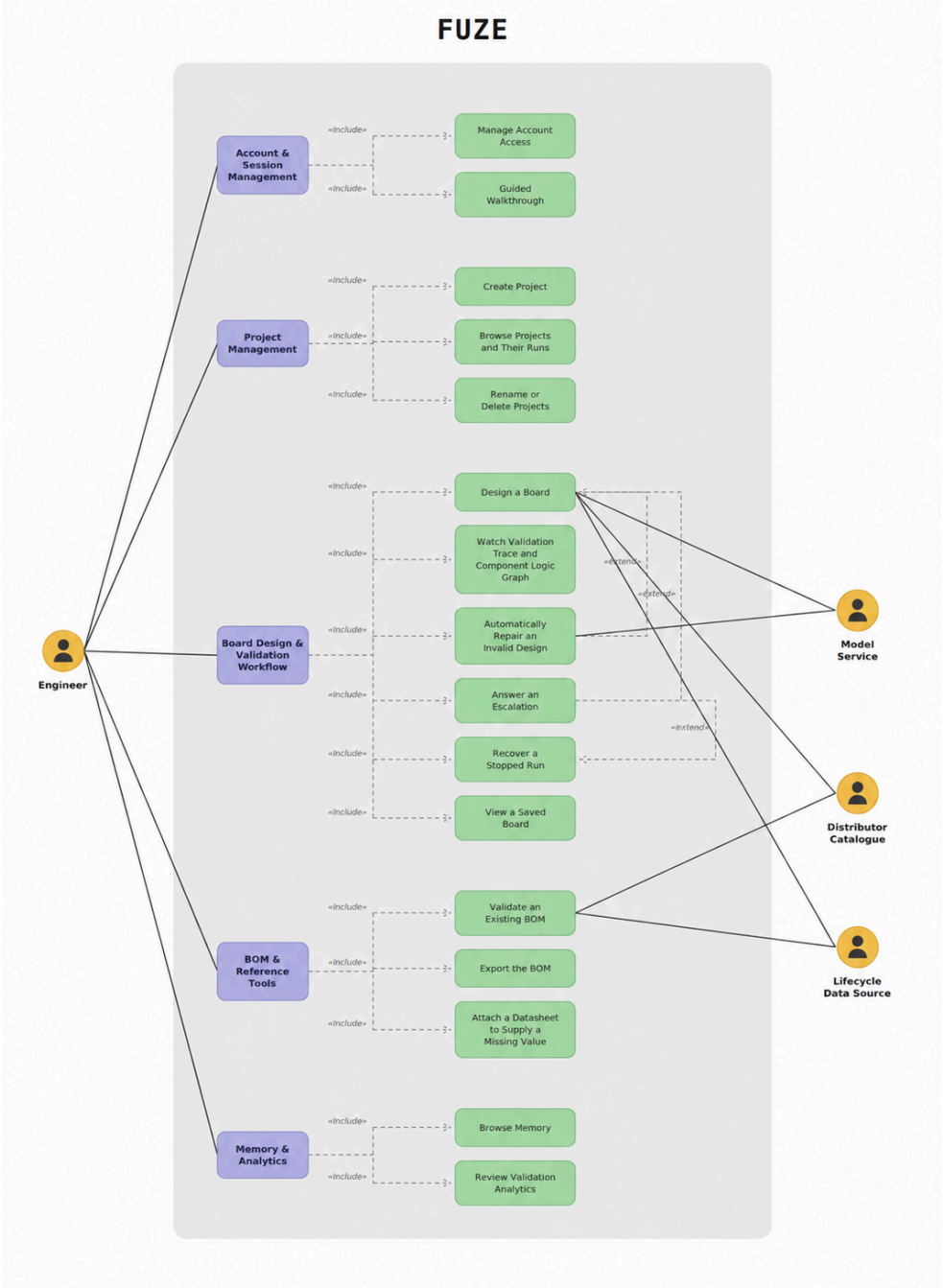


Figure 2: Use case diagram. The Engineer, with the Model Service, the Distributor Catalogue and the Lifecycle Data Source as external systems.

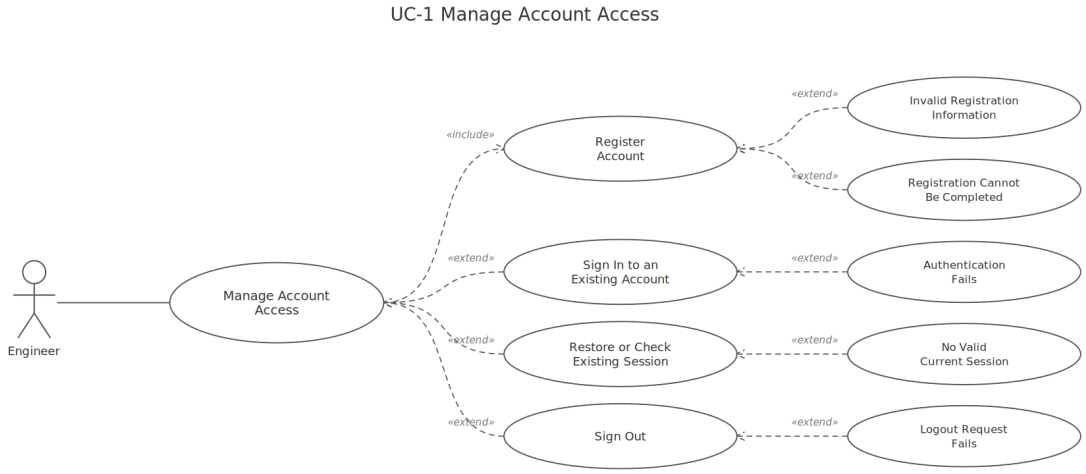
ID	Use case	Group
UC-1	Manage Account Access	Account and session management
UC-2	Create Project	Project management
UC-3	Browse Projects and Their Runs	Project management
UC-4	Rename or Delete a Project	Project management
UC-5	Guided Walkthrough	Account and session management
UC-6	Design a Board from a Plain-Language Brief	Board design and validation
UC-7	Watch Validation Trace and Component Logic Graph	Board design and validation
UC-8	Automatically Repair an Invalid Design	Board design and validation
UC-9	Answer an Escalation	Board design and validation
UC-10	Recover a Stopped Run	Board design and validation
UC-11	View a Saved Board	Board design and validation
UC-12	Validate an Existing BOM	BOM and reference tools
UC-13	Export the BOM	BOM and reference tools
UC-14	Attach a Datasheet to Supply a Missing Value	BOM and reference tools
UC-15	Browse Memory	Memory
UC-16	Review Validation Analytics	Memory and analytics

Table 1: Use Case List

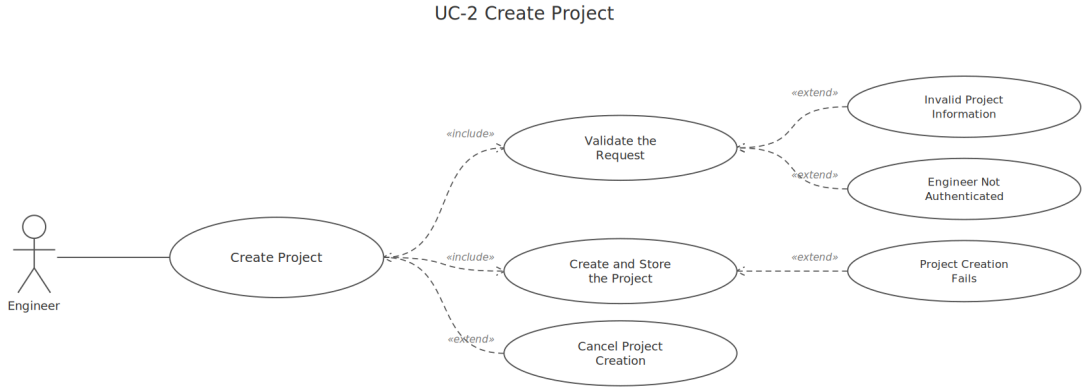
UC-6 is the central operation and the one on which the others depend. UC-7 observes what UC-6 and UC-8 produce and changes nothing. UC-8 runs inside a design run and is not started directly by the engineer. UC-9 and UC-10 are different recovery paths and must not be merged: UC-9 answers a workflow that paused itself at an interrupt, while UC-10 continues a run whose status is abandoned or error.

The overall use case diagram presents the main interactions within FUZE. The following subsections provide individual use case diagrams for each use case identified in Table 1.

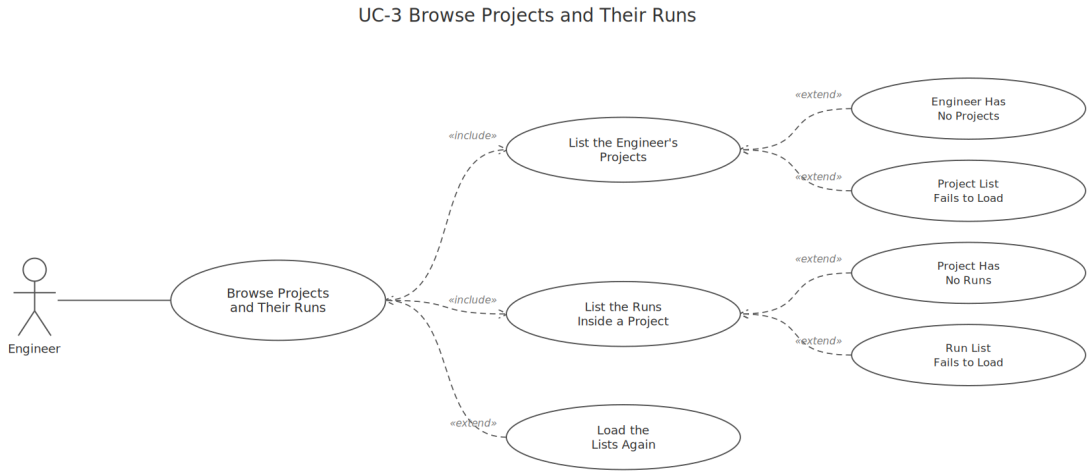
4.2.4.1 Use Case 1



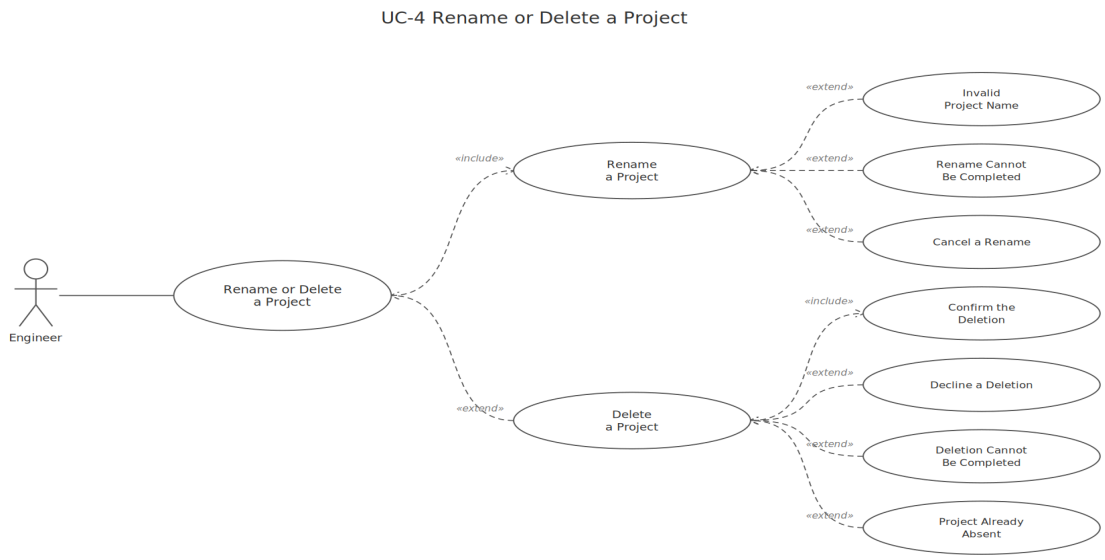
4.2.4.2 Use Case 2



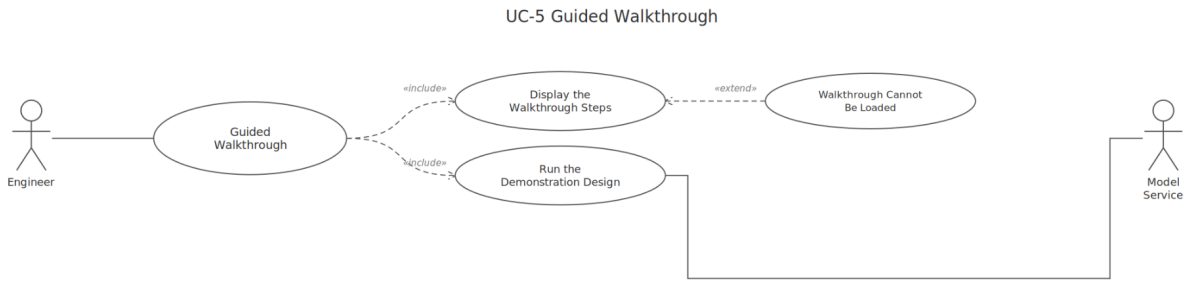
4.2.4.3 Use Case 3



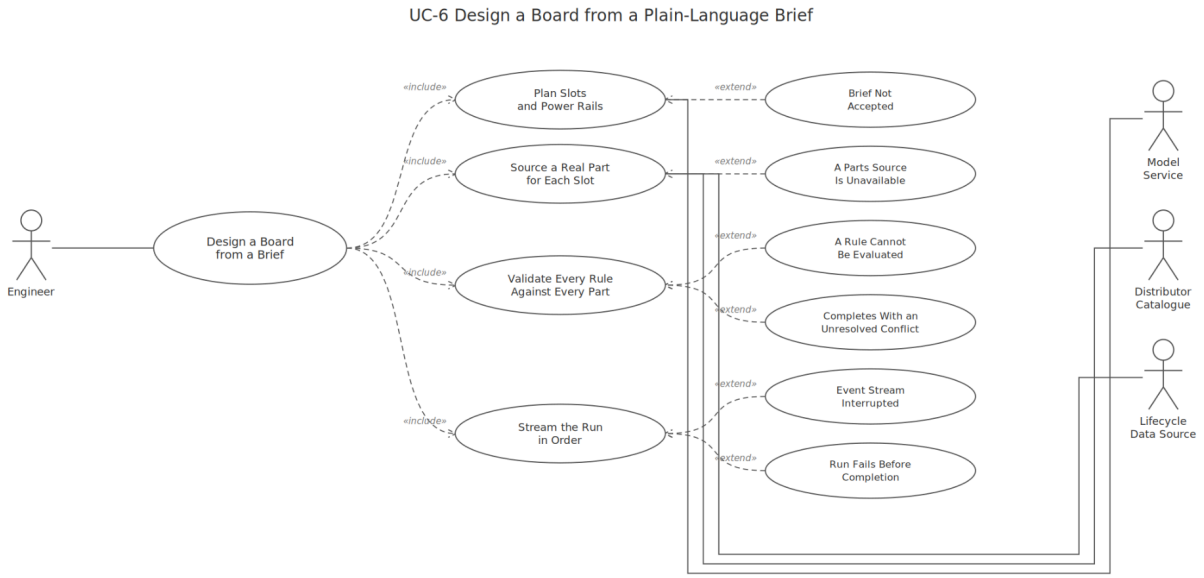
4.2.4.4 Use Case 4



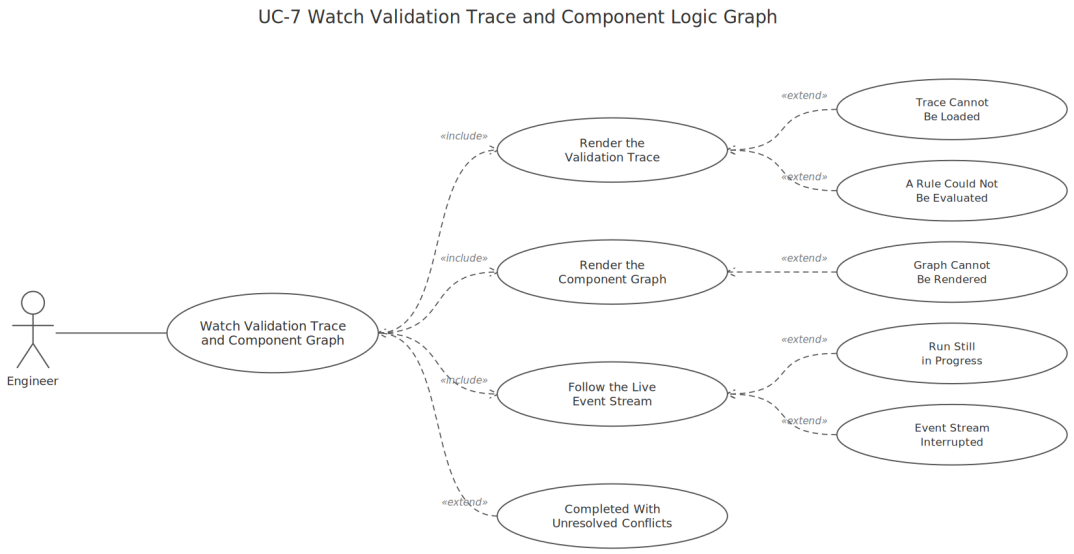
4.2.4.5 Use Case 5



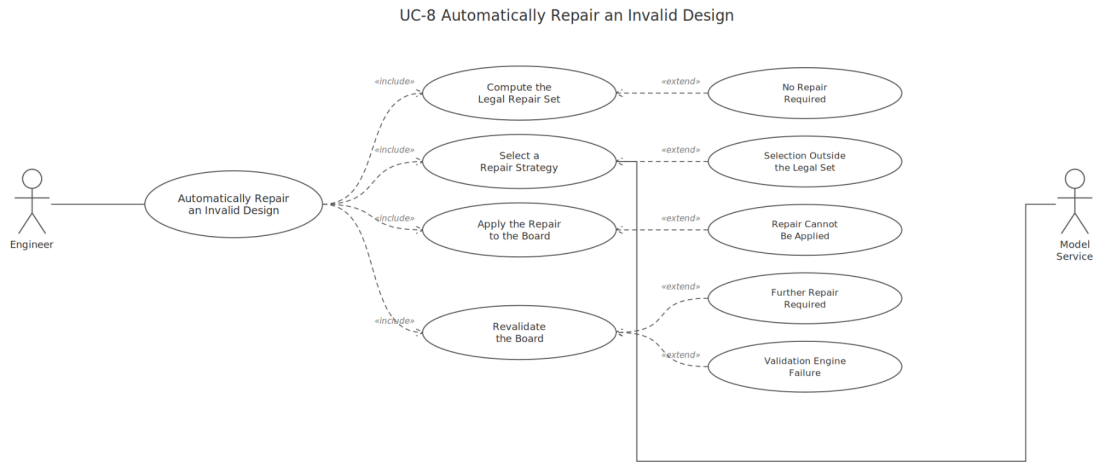
4.2.4.6 Use Case 6



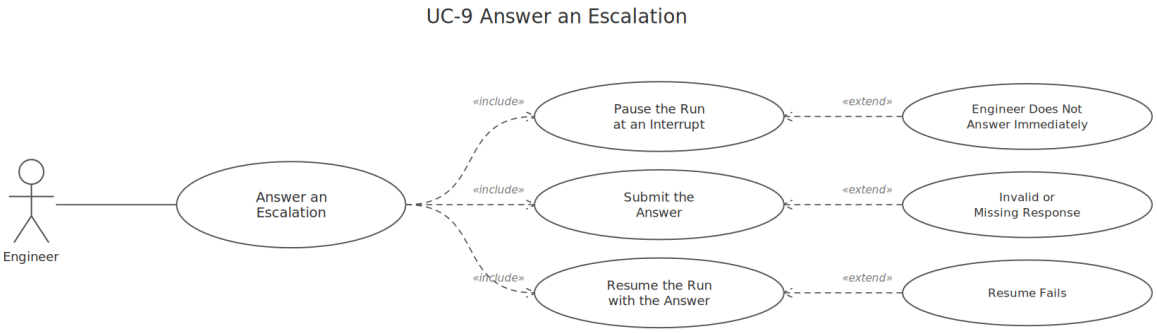
4.2.4.7 Use Case 7



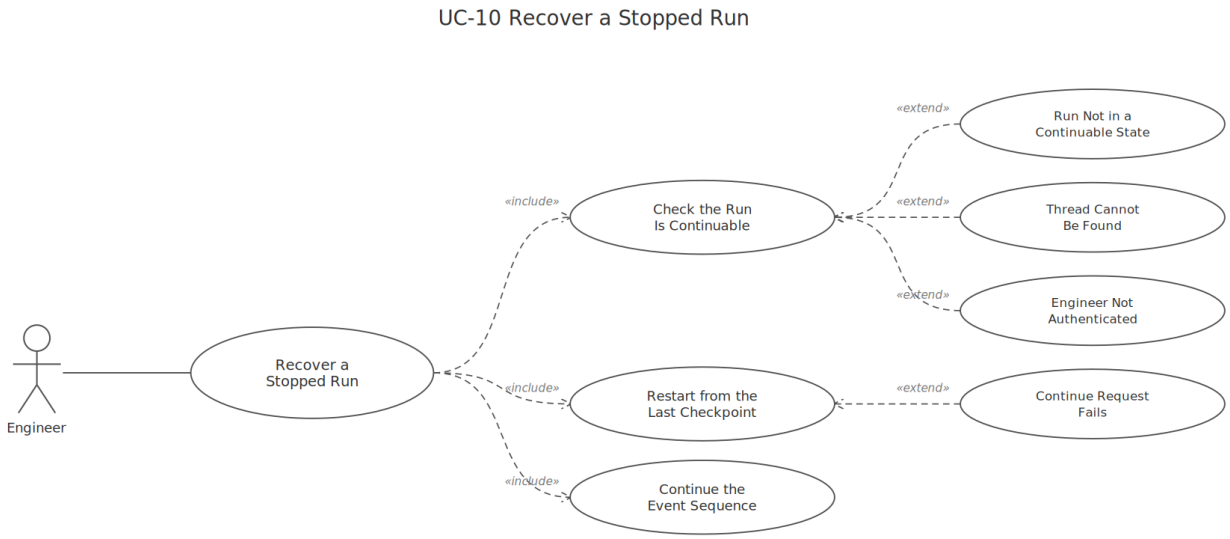
4.2.4.8 Use Case 8



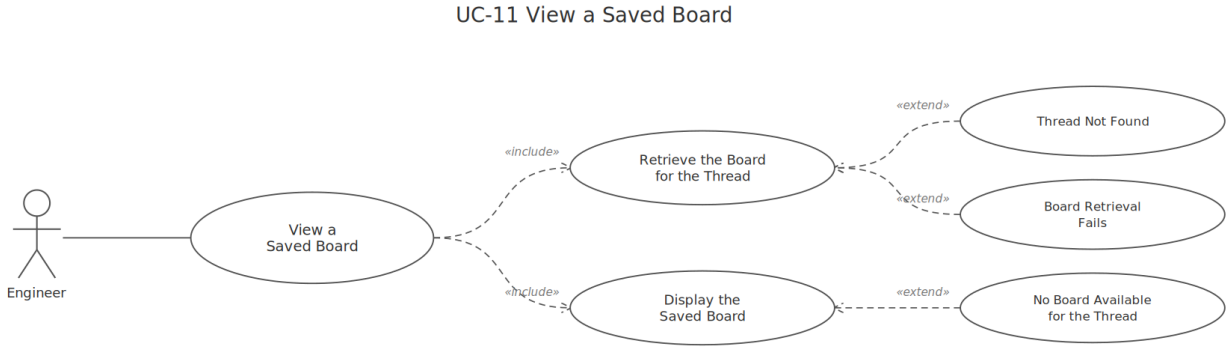
4.2.4.9 Use Case 9



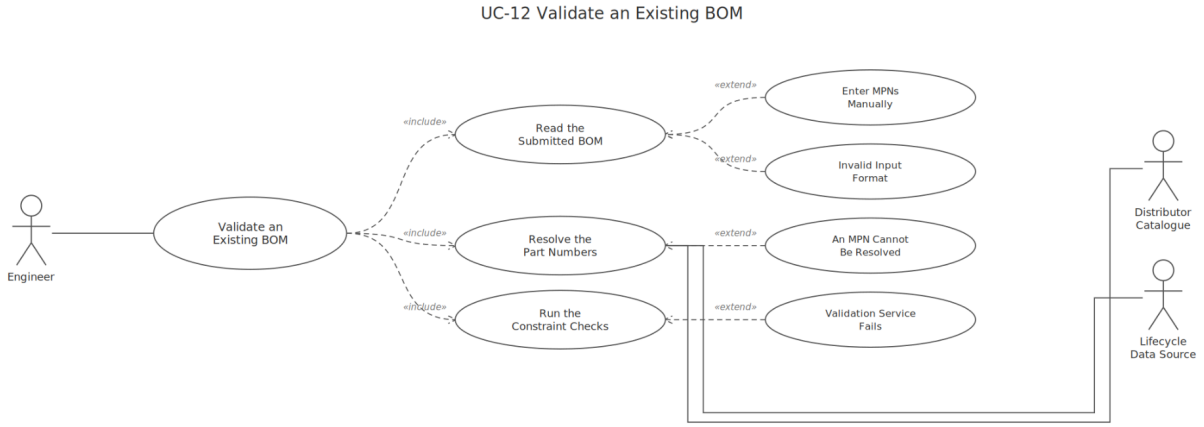
4.2.4.10 Use Case 10



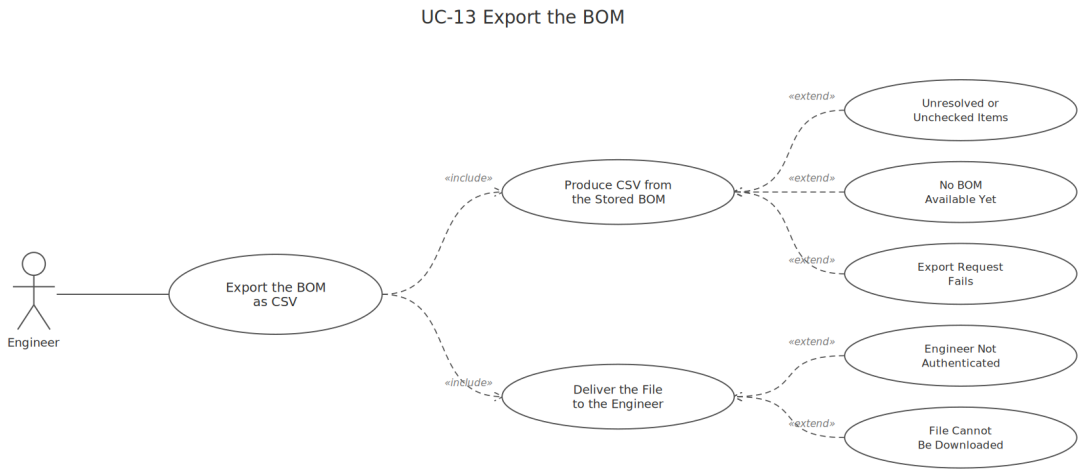
4.2.4.11 Use Case 11



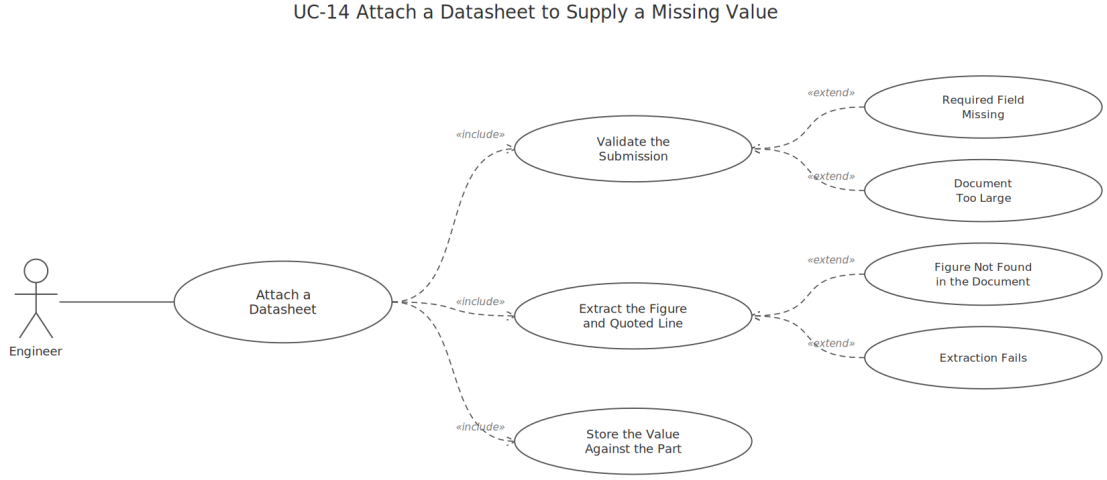
4.2.4.12 Use Case 12



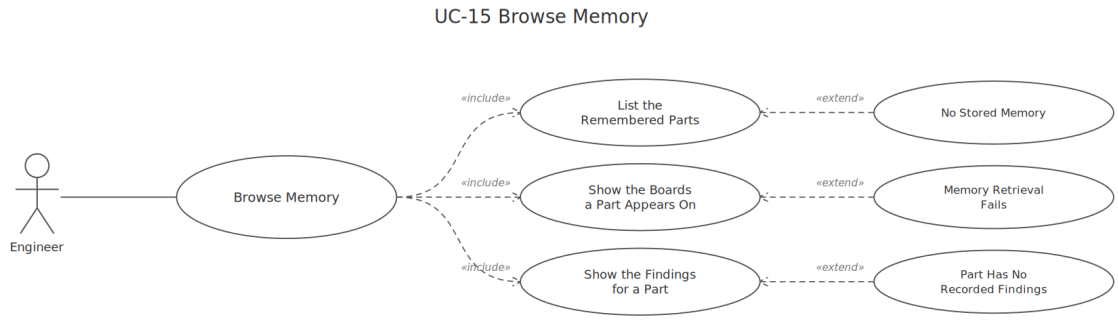
4.2.4.13 Use Case 13



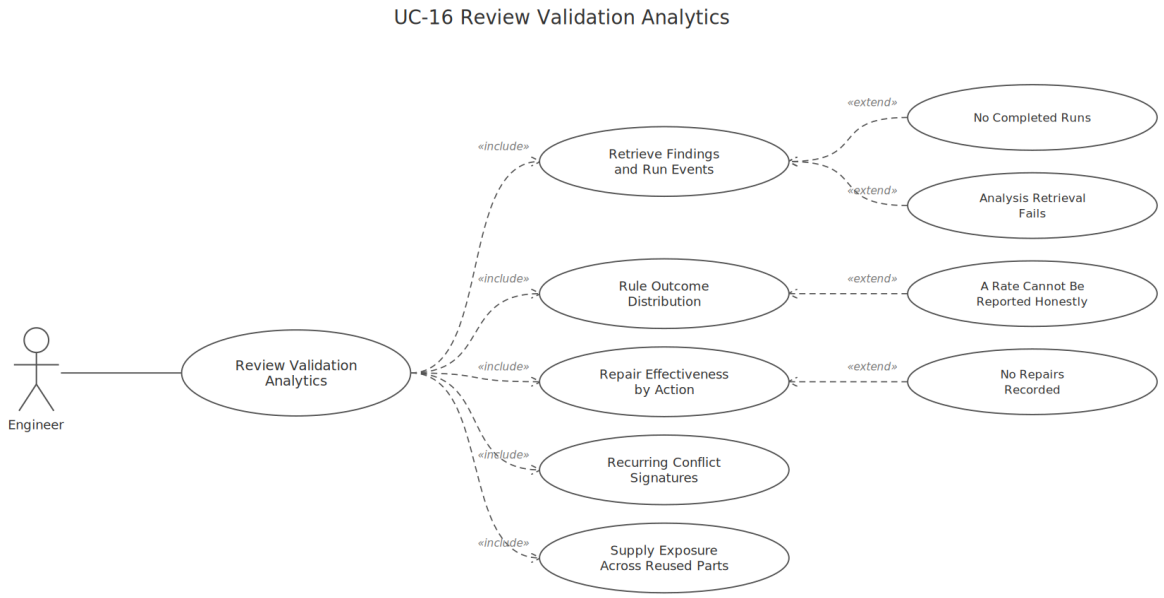
4.2.4.14 Use Case 14



4.2.4.15 Use Case 15



4.2.4.16 Use Case 16



4.2.5 Prototyping

A vertical slice is built before breadth. The slice runs a single small board from brief to validated bill of materials against live distributor data, using the real event contract, the real engine and the real interface. It is demonstrated at the fourth milestone. Everything after that milestone widens the slice rather than changing its shape. The purpose is to prove the contract between the seven workstreams early, while there is still time to change it.

4.2.6 Verification and Test Suites

Testing is organised as seven suites. Each suite has an owner and each target specification is evidenced by at least one of them.

Suite	What it covers	Evidences
Unit and property tests	Each rule in isolation, against hand-calculated arithmetic and against generated inputs.	DS-1, DS-2, DS-4
Regression board corpus	Forty recorded boards replayed against recorded distributor data	DS-2, DS-4, DS-7
Integration tests	Sourcing, normalisation, orchestration, persistence and the event stream together. Analysis queries verified against a seeded corpus, including the case where a withheld rate is required.	DS-5, DS-6, DS-9, DS-16
Adversarial and injection corpus	Twenty-five briefs written to make the system emit a false pass	DS-10, DS-14
End-to-end browser tests	The sixteen use cases driven through the real interface	DS-11, DS-15, DS-16

Suite	What it covers	Evidences
Usability sessions	Ten first-time participants, unassisted, timed	DS-15
Performance and availability	Latency, cold start, health response, sustained uptime	DS-5, DS-6, DS-12

Table 2: Test suites and the specifications they evidence

Backend coverage is held at 80 percent or above, and the constraint engine at 95 percent or above. Coverage is enforced in continuous integration as a gate rather than reported as a number.

Each target specification is traced below to the section of this document that defines it and to the suite that evidences it.

Specification	Defined in	Evidenced by
DS-1 Rule coverage	The constraint rule set	Unit and property tests
DS-2 Determinism	Determinism	Unit and property tests; regression board corpus
DS-3 Origin	Verdicts	Unit and property tests; regression board corpus
DS-4 No substituted defaults	Handling of missing data	Unit and property tests; regression board corpus
DS-5 End-to-end latency	Latency	Integration tests; performance and availability
DS-6 Streaming responsiveness	The event stream; streaming responsiveness	Integration tests; performance and availability
DS-7 Bounded repair	Bounded repair	Regression board corpus, using the ten unsatisfiable boards
DS-8 Test coverage	Verification and test suites	All suites, enforced as a gate in continuous integration
DS-9 Sourceability	Sourceability	Integration tests
DS-10 Model fencing	Model fencing	Adversarial and injection corpus
DS-11 Export fidelity	Export	End-to-end browser tests
DS-12 Availability	Availability and failover; Deployment Requirements	Performance and availability
DS-13 Security	Security	Integration tests
DS-14 Injection resistance	Model fencing	Adversarial and injection corpus
DS-15 Usability	Board design and validation workflow; The validation trace and the component graph	Usability sessions; end-to-end browser tests
DS-16 Validation Analytics	Validation analytics; Analytics integrity; Analysis	Integration tests; end-to-end browser tests

Table 3: Traceability of target specifications to this document and to the test suites

The scenarios and edge cases for each use case are set out below. A scenario is a path a test must drive. An edge case is a condition that has broken a comparable system before and that a passing test must therefore exercise deliberately.

Use case	Scenarios	Edge cases
UC-1 Manage Account Access	Register a new account; sign in to an existing account; restore a session on reload; sign out	Registration with a duplicate identifier; sign-in with wrong credentials; session check with no cookie; session check with an expired cookie; logout request fails at the server while the client has already cleared its state
UC-2 Create Project	Create a project and see it returned	Empty or over-length prompt; creation attempted while unauthenticated; server error mid-create; the same create submitted twice by a double click
UC-3 Browse Projects and Their Runs	List projects; open a project and list its runs	Account with no projects; project with no runs; list request fails and must not render as an empty list; list reloaded after a rename or delete; a project that was deleted in another tab
UC-4 Rename or Delete a Project	Rename and see the new name; delete after confirming	Name that fails validation; rename rejected by the server and the displayed name must revert; delete confirmed for a project that is already gone; delete declined at the confirmation step; delete of a project with runs in progress
UC-5 Guided Walkthrough	A new account completes the walkthrough and proceeds	Walkthrough fails to load and must not be recorded as completed; the demo design endpoint is slow or unavailable; the walkthrough is opened by an account that has already finished it
UC-6 Design a Board	Brief to validated bill of materials on a passing board; brief on a board that fails a rule and is repaired; brief that ends with an unresolved conflict	Empty brief; brief in twelve slots at the size limit; a rule that cannot be evaluated and must report unchecked; a distributor source unreachable mid-run; the stream breaking before completion; a brief naming a part that does not exist; two runs started in the same project at once
UC-7 Watch Validation Trace and Graph	Watch a live run; open the trace of a finished run; open a conflict and see every rule that ran on the failing part	Trace fails to load and must not render as no rules having run; graph fails while the trace is already displayed; stream interrupted mid-watch; a part with only unchecked verdicts; a board large enough to make the graph dense
UC-8 Automatically Repair an Invalid Design	A failure repaired on the first attempt; a failure needing a second attempt; a board that needs no repair	Repair that cannot be applied; engine unavailable for the re-check; the model returns an action outside the legal set; the model returns nothing in time; a slot the engineer named explicitly, where only swap, change_rail and escalate are permitted; the third repair on one slot, which must escalate

UC-9 Answer an Escalation	The run pauses, the engineer answers, the run continues	Answer that does not supply what was asked; the engineer leaves without answering and returns later; resume fails after a valid answer was submitted; two answers submitted for the same interrupt
UC-10 Recover a Stopped Run	Continue a run whose status is abandoned; continue a run whose status is error	Continue attempted on a run that is paused at an interrupt, which must be refused; continue on an unknown thread id; continue on a thread belonging to another engineer; event sequence numbers must carry on from the stored high-water mark rather than restart
UC-11 View a Saved Board	Open a finished run and see its saved board	Thread exists with no saved board; thread not found; retrieval fails part way and must not render partial data as the saved board; opening a board must not make the run active
UC-12 Validate an Existing BOM	Validate a CSV upload; validate pasted manufacturer part numbers	File that is not valid CSV; CSV with missing or unexpected columns; a part number that cannot be resolved, which must not be reported as passing; duplicate part numbers in one submission; a file large enough to hit the upload limit; a submission where every entry is unresolvable
UC-13 Export the BOM	Export a finished bill of materials as CSV and open it in a spreadsheet	Run with no bill of materials yet, where no empty file may be produced; a bill of materials with unresolved slots and unchecked rules, which must be represented honestly; export by an engineer who cannot access the run; the file is produced but the download fails
UC-14 Attach a Datasheet	Attach a datasheet and see a previously unchecked rule evaluate and cite it	Missing part number, package or file; file over the size limit; a document with no usable thermal figure; a document that cannot be parsed; a scanned datasheet with no extractable text; a figure found but with no line to quote
UC-15 Browse Memory	Open memory, select a part, see the boards it appears on and the findings recorded for it	Memory with nothing in it, which must show an empty state and not an error; a part with no recorded findings; retrieval fails; browsing must leave every record unchanged
UC-16 Review Validation Analytics	Open the analysis and see rule outcomes, repair effectiveness, recurring conflict signatures and supply exposure across completed runs	An account with no completed runs, which must show an empty state and not an error; a rule whose unchecked evaluations exceed the threshold, where the rate must be withheld and the reason named; completed runs containing no repairs, where repair effectiveness has nothing to report; retrieval fails; runs belonging to another engineer must not appear; abandoned and errored runs must be excluded from every count

Table 4: Test scenarios and edge cases by use case

5 Constraints

5.1 Distributor Rate Limits

The distributor interfaces used for component data are rate limited. Search queries must push numeric constraints into the request rather than fetching broadly and filtering afterwards, and repeatable testing must run against recorded data rather than live calls.

5.2 Data Source Coverage

No single source carries everything the engine needs. The JLCPCB parts library supplies component parameters, stock and price. Mouser supplies lifecycle status and datasheet links. Manufacturer datasheets are used where catalogue data is incomplete. Nexar was considered and rejected as a primary source because its free tier is too limited for the volume of queries a run makes.

5.3 Missing Thermal Data

Thermal resistance is not published for every part. Where it is absent, the system uses recognised reference data and names the reference it used, or reports the rule as unchecked. It never assumes a figure.

5.4 Missing Duty Cycle

Duty cycle is often unavailable. Thermal checks therefore use conservative assumptions, and the assumption used is stated in the verdict rather than hidden in it.

5.5 Streaming and Scaling

Server-sent event streams do not scale across workers without additional infrastructure. The delivered system runs a single application worker. Horizontal scaling is outside the current scope.

5.6 Free-tier Hosting

The hosting tier sleeps when idle. Cold start delay must be accounted for and mitigated during the demonstration period.

5.7 Model Availability by Region

Access to the language model endpoint depends on region. The endpoint and its configuration must be set to a compatible region.

5.8 Project Schedule

The project runs for fourteen weeks in six phases, beginning 20 August 2026 and completing 25 November 2026. The interface contract is frozen in week five, which is what allows the engine, the interface and the test suites to proceed in parallel. The engine is complete and verified in week eight, the vertical slice runs on live data in week eleven, and every target specification is evidenced by week thirteen.

5.9 Open Issues

ID	Open issue	Raised in	Owner
OI-1	Whether registration signs the engineer in automatically or returns them to the sign-in screen	UC-1	Frontend and backend developers
OI-2	Whether project and run lists need paging, sorting or searching once an engineer has many projects	UC-3	Frontend and backend developers
OI-3	Whether deleting a project also removes the runs recorded against it and anything retained from those runs	UC-4	Backend developer
OI-4	Whether a run continues on the server after the engineer disconnects	UC-6	Backend developer
OI-5	Whether an engineer can cancel a run that is in progress	UC-6	Frontend and backend developers
OI-6	Whether the trace and the component graph can be hidden independently of each other	UC-7	Frontend developer
OI-7	The exported CSV does not yet carry a per-row verdict or a list of rules that could not be evaluated for that row	UC-13	Frontend and backend developers
OI-8	How duplicate manufacturer part numbers are handled during standalone validation	UC-12	Backend developer
OI-9	Whether findings from standalone bill of materials validation are retained and visible through memory browsing	UC-12, UC-15	Backend developer
OI-10	Whether the saved board route always returns the latest persisted board state	UC-11	Backend developer
OI-11	The response when a thread exists but holds no persisted board	UC-11	Backend developer
OI-12	The exact component identifier accepted when attaching a datasheet	UC-14	Backend developer
OI-13	How multiple candidate datasheets or component variants are handled	UC-14	Backend developer
OI-14	Whether the datasheet route returns a file, a reference, extracted values, or a combination	UC-14	Backend developer

OI-15	The use case diagram names UC-10 “Resume a Paused Run” while the use case description names it “Recover a Stopped Run” and restricts it to abandoned and error runs. This specification follows the description	Diagram and UC-10	Lead developer
OI-16	The ambient temperature used by the thermal rule is not defined anywhere. A fixed assumption would be a substituted default, which DS-4 forbids, so either the brief must capture it or the rule reports unchecked	R3, DS-4	Lead developer
OI-17	Converter efficiency, needed by the switching branch of the thermal rule, is not a published catalogue field for most parts	R3, DS-4	Lead developer
OI-18	The build quantity that stock is compared against is not defined, and the brief has no place to state it	R6, DS-4	Lead developer
OI-19	Memory browsing is implemented and shipping. Its status as delivered scope should be confirmed with the sponsor and the proposal amended to record it.	UC-15	Project manager
OI-20	The guided walkthrough and standalone bill of materials validation are implemented and shipping. Neither appears in the proposal's target specifications or deliverables. Their status as delivered scope should be confirmed with the sponsor and the proposal amended to record them.	UC-5, UC-12	Project manager
OI-21	When the relax_requirement repair action may be applied, and whether the engineer is told that a stated requirement was weakened	UC-8	Lead developer
OI-22	The threshold above which a rate is withheld rather than reported is proposed at twenty percent of the denominator and has not been confirmed against real run data	UC-16	Lead developer

Table 5: Open issues carried forward from the source documents

5.10 Assumptions

Analysis volume. The analysis is meaningful only over a body of completed runs. A new account has none, and the interface presents an empty state rather than figures computed over one or two runs. Demonstration requires a seeded corpus.

6 Operational Requirements

6.1 Support

Support during the demonstration period is provided by the team rather than by a help desk. The project manager is the single point of contact for the sponsor. Technical queries are answered within three working days, and a query that goes unanswered states the assumption the team will proceed under.

6.2 Application Services and Technical support

The sponsor holds read access to the repository and the project board from week one and may observe progress at any time without asking. The release engineer owns continuous integration, environment and secret management, database migrations, monitoring and the runbook. The lead developer reviews every change that touches the engine or the interface contract.

6.3 Administration Features

Access is per engineer. An engineer sees only their own projects, runs, boards and bills of materials. Every operation that reads or changes stored state checks the authenticated identity first, and a request from an engineer who does not hold the resource is refused rather than answered with empty data.

6.4 Availability and Failover

The deployed system maintains at least 99 percent availability across the two-week demonstration period. Cold start completes within 60 seconds and a warm health check responds within 500 milliseconds. A documented local-hosting fallback exists so that a demonstration can proceed if the hosted environment is unavailable.

6.5 Audit Trail

Every run is recorded against a project with its brief, its events in order, its verdicts, its repairs and its final board. A verdict cannot be recorded without the field, the value and the source that produced it. Escalations and the answers given to them stay attached to the run they belong to.

7 Functional Requirements

FUZE is a self-service web application. An engineer works through a browser, and every function below is reachable by an authenticated engineer acting on their own data.

7.1 Account and Session Management

An engineer can register an account, sign in, have an existing session restored when the application is reopened, and sign out. Functionality that depends on authentication is unavailable without it. A failed sign-in leaves the engineer unauthenticated and creates no session. A failed sign-out is not shown to the engineer as a successful one.

A new account is offered a guided walkthrough that introduces the design workflow using a demonstration run. A walkthrough that fails to load is not recorded as completed.

7.2 Project Management

An engineer can create a project, list the projects held for them, open a project and list the runs inside it, rename a project, and delete one. Listing reads data and changes nothing. A list that fails to load is shown as a failure rather than as an empty list, because the two mean different things to the engineer. Deletion is confirmed before it is sent, and a rename or deletion that the server did not confirm is never displayed as saved.

7.3 Board Design and Validation Workflow

An engineer describes a board in plain language and submits it. The system converts the description into component slots and power rails, fills each slot with a real part, and re-runs every rule against every part after every placement. Progress is streamed to the browser in order as the run proceeds.

Where a rule fails, the engine determines which parts may be changed, the model selects one change from that set, the engine applies it, and every rule runs again on the result. A slot is repaired at most three times. Beyond that the conflict is raised to the engineer as an explicit trade-off rather than repaired further.

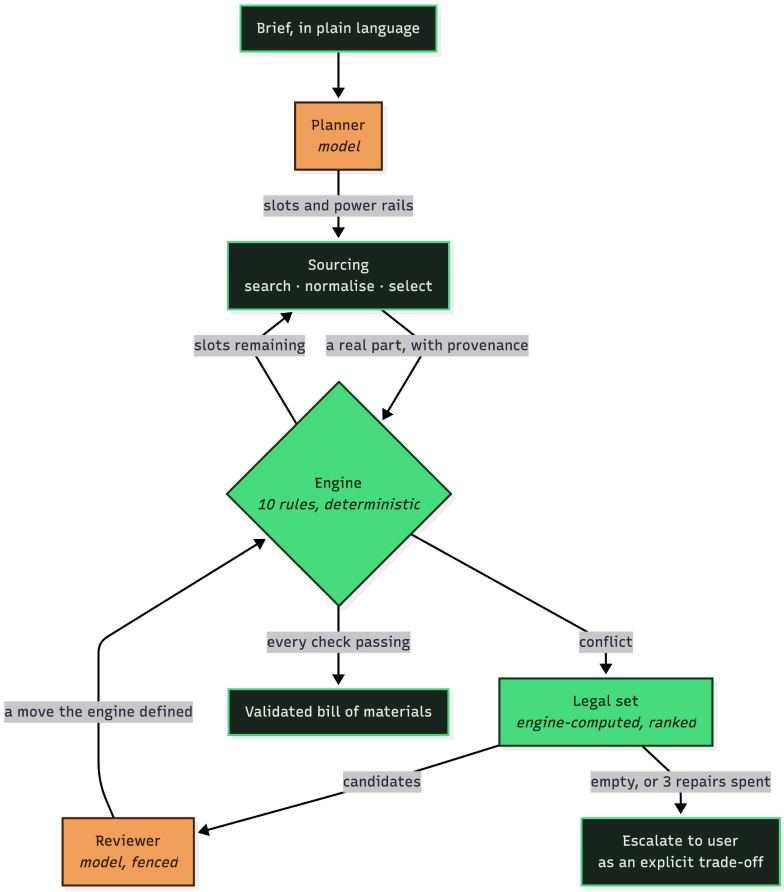


Figure 3: Validation and bounded repair loop, using a legal set of repair actions computed by the engine.

Where an engineer accepts a conflict rather than repairing it, the acceptance is recorded against that slot and rule. On revalidation the rule still runs and still reaches the same conclusion; its verdict is recorded as a warning, and the acceptance that produced the downgrade is carried with it. An acceptance never converts a failure into a pass, and it never suppresses a rule.

Where the workflow cannot safely decide how to proceed, it pauses itself and asks the engineer a question. The run stays paused until an answer is supplied, and the engineer can leave and come back to it. A run that stopped because the client disconnected or because it failed is continued from its last completed checkpoint rather than restarted from an empty board.

An engineer can watch the validation trace and the component logic graph while a run is streaming or after it has finished. Watching changes nothing. A conflict is displayed together with every rule that ran on the failing part, including the rules that passed, so that the engineer can see the whole picture rather than only the failure.

An engineer can reopen a finished run and view its saved board without making the run active.

7.4 The Constraint Rule Set

The engine implements ten rules. Each is a pure function from a board to a list of verdicts. Each rule holds no network client, no model client and no read of the system clock. Each verdict names the rule, the part it ran on, the outcome, the field that decided it, the value read from that field, and the source that value came from.

A rule returns one of four outcomes. A pass means the relation held. A fail means the relation did not hold. A warning means the condition is worth the engineer’s attention but does not block completion. Unchecked means the rule could not be evaluated because a value it needs is not available; the rule names the missing field and substitutes nothing. Unchecked is never reported as a pass.

Every rule runs against every part after every placement, because a part that was acceptable before a neighbour was added may not be acceptable after.

7.4.1 R1 voltage_overlap

Detects a component receiving a supply voltage outside the range it supports.

Inputs are the part’s minimum and maximum supply voltage and the nominal voltage of the rail the part sits on. The relation is part.vmin ≤ rail.v ≤ part.vmax. The rule fails if the relation does not hold and is unchecked if either voltage limit is unpublished or if the part is not assigned to a rail.

A sensor supporting 2.7 V to 3.6 V placed on a 5.0 V rail fails, citing the maximum supply voltage field, the value 3.6 V, and the catalogue entry it was read from.

7.4.2 R2 current_budget

Detects a rail that is asked for more current than its source can supply.

Inputs are the peak current of every part on the rail and the maximum output current of the rail’s source. A margin of 15 percent is applied. The rule fails if Σ i_peak × 1.15 > source.i_max. It is unchecked if the

peak current of any part on the rail is unpublished, and it names the part and the field that is missing rather than treating the unknown draw as zero.

A 600 mA regulator feeding a radio at 500 mA and a display at 200 mA fails: 700 mA with margin is 805 mA against a 600 mA source.

7.4.3 R3 thermal_dissipation

Detects a regulator that would run past its safe operating temperature.

For a linear regulator, dissipation is P = (vin − vout) × I. For a switching regulator it is P = vout × I × (1/η − 1), where η is the conversion efficiency. The temperature rise is ΔT = P × θJA, and the rule fails if ambient + ΔT > part.temp_max.

Thermal resistance is taken from the part's datasheet where it is published, from a named reference table for the package where it is not, or from a datasheet the engineer attaches. The source used is named in the verdict. Where it cannot be obtained from any of those, the rule is unchecked.

Where duty cycle is unavailable, continuous operation is assumed and the assumption is stated in the verdict.

A linear regulator converting 5 V to 3.3 V at 400 mA dissipates 0.68 W. In a package with a thermal resistance near 200 °C/W this is a rise of about 136 °C, which at 25 °C ambient puts the junction past the rating of a part limited to 150 °C.

7.4.4 R4 interface_role_match

Detects a peripheral needing an interface that no controller on the board supports.

Inputs are the interfaces each part exposes and the role it takes on each. A match requires the same protocol family, complementary roles, and compatible signal levels or a level translator present on the board. The rule fails when no compatible pairing exists between a peripheral and a controller, and is unchecked when the interface list for either part is unpublished.

A sensor exposing only SPI on a board whose microcontroller exposes only I²C fails.

7.4.5 R5 pin_budget

Detects a microcontroller being asked for more pins than it has.

Inputs are the pins each peripheral requires and the pins the microcontroller has available after fixed functions are deducted. The rule fails if Σ peripheral.pins_required > mcu.pins_available. A shared bus is counted once for the bus rather than once per device on it. The rule is unchecked if either pin count is unpublished.

7.4.6 R6 availability

Detects insufficient stock or an unsuitable lifecycle status.

The rule fails if distributor stock is below the minimum the build requires. It warns if lifecycle status is not-recommended-for-new-designs or obsolete, because such a part will work but should be a deliberate choice. It is unchecked if no source returns a stock figure.

Stock changes over time, which would otherwise break determinism. Stock and lifecycle are read once when the part is selected, frozen into the run's part record, and cited with the time of that read. A replay of the same run against the same recorded data therefore produces the same verdict.

7.4.7 R7 temperature_rating

Detects a component that cannot operate across the temperature range the application requires.

The rule passes only when part.temp_min ≤ required_min and part.temp_max ≥ required_max. It fails when the part's range does not cover the required range, and is unchecked when the part's range is unpublished or when the brief states no required range.

7.4.8 R8 footprint

Detects a package unsuited to the intended assembly process.

This rule warns and never fails. A fine-pitch or ball-grid package on a board intended for hand assembly is worth flagging but is not an electrical fault, and a warning does not block completion of the run.

7.4.9 R9 energy_budget

Detects a power source that cannot support the required operating time.

The rule fails if capacity_mAh / continuous_draw_mA < required_runtime_h. It applies only when the brief states both a battery and a required runtime. Where either is absent the rule is unchecked, and it does not invent a runtime in order to have something to check.

7.4.10 R10 rail_coverage

Detects a component that is not connected to any power rail.

A part with no rail is reported as unchecked and never as a pass. This rule exists to make sure the voltage and current rules are not silently skipped: without it, a part that no rule can reach would produce no verdicts at all and would look clean.

Two advisory thresholds sit alongside the ten rules and produce warnings rather than verdicts. A part operating above 80 percent of a rated limit is flagged as approaching derating. A calculated temperature rise above 60 degrees Celsius is flagged for the engineer's attention. Neither threshold blocks completion, and neither is a rule: they do not appear in the rule registry and they produce no pass or fail.

7.5 Bill of Materials and Reference Tools

An engineer can validate a bill of materials they already hold, independently of the design workflow, by uploading a CSV file or pasting one manufacturer part number per line. A part that cannot be resolved is identified in the results and is never shown as having passed.

An engineer can export the bill of materials of a run as a CSV file. The export reflects the bill of materials the system holds for that run. Unresolved slots and unchecked rules are represented honestly in the file and are not written as passes. Where a run has no bill of materials, no file is produced.

An engineer can attach a manufacturer datasheet to a component whose thermal resistance is not published anywhere. The system extracts the figure and keeps the line it was read from, so that a rule which previously reported unchecked can evaluate and cite the datasheet. Where no usable figure is found, nothing is stored and the rule stays unchecked.

7.6 Memory

An engineer can browse what the system has retained from earlier design activity: the parts it remembers, the boards those parts appear on, and the findings recorded against them. Browsing changes nothing, and an empty memory is shown as an empty state rather than as an error.

Memory is also read by the system during a run. When a rule fails, the engine computes a signature for the situation from the rule that failed and the banded values that caused it. The system retrieves up to three earlier repairs recorded against that same signature which were applied and which subsequently held and supplies them to the model as context when it selects a repair strategy. A precedent informs the model's choice. The selection is still checked for membership in the engine-computed legal set before anything is applied, so a precedent cannot produce an illegal repair, and a repair a precedent suggested is revalidated by the same rules as any other. Where no precedent exists for a signature, the model selects from the legal set unaided.

7.7 Engineer Identity

An engineer is identified by the account they register. Every request that reads or changes stored state carries the authenticated identity, and the system resolves that identity before it acts. An engineer holds their own projects, runs, boards and bills of materials, and nothing outside that set is reachable.

7.8 The Brief

The primary input is a plain-language description of a board, written in English. It is free text. From it the system derives the component slots, the power rails, and any stated requirement such as an operating temperature range, a battery capacity or a required runtime. A brief that is empty or fails validation starts no run.

A requirement stated in the brief becomes a constraint the engine can check. A requirement not stated leaves the corresponding rule unchecked rather than defaulted, which is why the brief is treated as an input to the rule set and not only to the planner.

7.9 Bill of Materials Upload

A bill of materials may be supplied for standalone validation either as a CSV file or as text containing one manufacturer part number per line. Input that cannot be read in a supported format starts no validation. The supported CSV structure is an open issue.

7.10 Datasheet Upload

A manufacturer datasheet may be supplied as a PDF together with the manufacturer part number and the package. All three are required, and the document is subject to a size limit. A request missing any of the three is rejected and stores nothing.

7.11 Component Data

Component parameters, stock and price are read from the JLCPCB parts library. Lifecycle status and datasheet links are read from Mouser. Manufacturer datasheets and web search supply values the catalogues do not carry. Every value read is kept together with the field it came from and the source it was read from, from the distributor payload through normalisation and into the engine.

7.12 Validation Analytics

An engineer can review how the system has behaved across their completed runs. Four views are reported: the outcome distribution of each rule, showing evaluations that passed, failed and could not be evaluated; the proportion of each repair action that held after revalidation; the conflict situations that recur most often and the repair that most often held for each; and the lifecycle status mix and single-source count across the parts appearing in the engineer's boards.

Every figure is computed at the time it is requested from records the system already holds, and no figure is retained as a stored score. A verdict depends on the conditions of the board that produced it, so the analysis reports where a finding occurred and under what conditions, and never rates a part independently of the board it was used on. Analysis covers only the runs belonging to the authenticated engineer. Runs that were abandoned or that ended in error are excluded. Viewing the analysis changes nothing.

8 Process Requirements

8.1 Interface Surface

The application server exposes a JSON interface over HTTPS. One route streams; the rest are request and response. The routes below are those named in the use case descriptions. Routes still to be assigned are marked and carried in the open issues table rather than invented here.

Method and route	Purpose	Use case
POST /auth/register	Create an account	UC-1
POST /auth/login	Establish a session	UC-1

POST /auth/logout	End the current session	UC-1
GET /auth/me	Return the currently authenticated engineer	UC-1
POST /projects	Create a project	UC-2
GET /projects	List the engineer’s projects	UC-3
GET /projects/{id}/threads	List the runs inside a project	UC-3
PATCH /projects/{id}	Rename a project	UC-4
DELETE /projects/{id}	Delete a project	UC-4
POST /design	Submit a brief and start a run; the response is a server-sent event stream	UC-6
POST /design/demo	Run the demonstration design used by the guided walkthrough	UC-5
POST /resume	Supply an answer to a run paused at an interrupt; body carries the thread identifier and the answer	UC-9
POST /threads/{id}/continue	Continue a run whose status is abandoned or error, from its last checkpoint	UC-10
GET /threads/{id}/board	Return the saved board for a run	UC-11
POST /datasheet	Attach a datasheet and extract the missing figure	UC-14
POST /bom/validate	Validate an uploaded or pasted bill of materials	UC-12
GET /export/{thread_id}.csv	Export the bill of materials as CSV	UC-13
GET /memory	Browse retained memory	UC-15
GET /analytics	Return validation analytics across the engineer's completed runs	UC-16

Table 6: Interface Surface

Four conventions apply across the interface. Every route except registration and sign-in requires an authenticated session and refuses the request without one. A request for a resource the engineer does not hold is refused rather than answered with empty data, so that absence and denial are never confused. A request that fails validation is rejected before any state changes. An error response carries a machine-readable code and a message, and the application never presents a failed operation as a successful one. The conditions below are those the use case descriptions require the interface to distinguish. Every response is implemented and is covered by a test.

Condition	Response	Source
Request fails validation, such as a datasheet submitted without a part number, package or file	422	Named in UC-14

Run is not in a continuable state, such as a continue request against a run paused at an interrupt	Conflict	Named in UC-10
No authenticated session	401 Unauthorized	Required by UC-1 through UC-4, UC-6, UC-10, UC-13
Resource exists but is not held by this engineer	404 Not Found	Required by UC-3, UC-10, UC-11, UC-13; absence and denial must not be distinguishable to a caller who does not hold the resource
Thread or project not found	404 Not Found	Required by UC-4, UC-10, UC-11
Upload exceeds the size limit	422 Unprocessable Entity	Required by UC-14
Unexpected server failure	500 Internal Server Error	Required throughout; the operation must not be presented as succeeded

Table 7: Error conditions the interface must distinguish

8.2 The Event Stream

A design run streams its progress. The stream carries the plan, each slot as it is filled, the part selected with the origin of its values, each verdict as it is produced, each repair as it is applied, any escalation, and the completion of the run. Verdicts that pass and verdicts that are unchecked are streamed alongside verdicts that fail, because the interface must be able to show every rule that ran on a part and not only the rule that broke.

Every event carries a sequence number that increases by one within a run. The server keeps a high-water mark for each run. A client that reconnects supplies the last sequence number it saw and receives what follows. A run continues after an abandoned or failed state carries on from the stored high-water mark rather than restarting its numbering, so that no frame is discarded by the client as a duplicate.

The events below are those the use case descriptions require the stream to carry. They are stated here so that the contract can be frozen at the second milestone. Event names and the field-level schema are confirmed at that milestone; what each event must carry is fixed by the use cases and is not open.

Event	Emitted when	Must carry
Run accepted	The brief is accepted, before any external call is made	Run and thread identifiers, sequence number
Plan ready	The brief has been converted into slots and rails	The slot list and the rail list
Slot sourcing	Sourcing begins for a slot	Slot identifier
Part selected	A real part is chosen for a slot	Slot identifier, manufacturer part number, the typed part specification, and for every value in it the field name and the source it was read from
Verdict	A rule finishes against a part	Rule, part, outcome, the field that decided it, the value read, the source; and for an unchecked outcome, the name of the missing field

Repair proposed	The model selects an action from the legal set	Slot, the action selected, the legal set it was selected from, and the repair count for that slot
Repair applied	The engine applies the selected action	Slot, the action, and the change made to the board
Escalation	The run pauses at an interrupt, or the repair limit for a slot is reached	Thread identifier and either the question asked or the trade-off stated
Bill of materials ready	Every slot is filled and every rule has run	The bill of materials with price and stock for each item
Run completed	The run finishes	Final status
Reasoning	A line of plain-language narration	The workflow reaches a point where an engineer watching the run would otherwise see an unexplained pause, such as normalisation of a shortlist, a rescue search, or the thinning of a candidate set
Run error	The run fails	Error code and message
Heartbeat	No event has been sent for the keep-alive interval	Nothing; it carries no sequence number and is not part of the sequence

Table 8: Events the stream must carry

Passing and unchecked verdicts are streamed as events in their own right and not folded into a summary at the end, because the interface must be able to show every rule that ran on a part while the run is still in progress.

8.3 Determinism

The engine produces the same verdicts for the same board and the same part data on every run. It holds no network client, no model client and no read of the system clock. Values that change over time, principally stock and lifecycle status, are read once at selection time, frozen into the run’s part record, and cited with the time of that read. Where a language model is used to normalise free-text catalogue values, it runs before the engine and outside it: its output is a typed part specification that is stored with the run, and the engine reads only that. Replaying a run therefore replays the normalised values rather than re-deriving them.

8.4 Handling of Missing Data (DS-4)

No rule substitutes a default. The mechanism is in the type rather than in the discipline of the person writing the rule.

Every field on a part specification that a rule reads is optional and holds either a value or nothing. Alongside the value the specification carries the verbatim distributor parameter it was derived from and the name of the field it was read from, so a verdict can cite its origin. A rule tests for the presence of each field it needs before using it. Where a required field is absent, the rule returns an unchecked verdict

naming that field and that part, and evaluates no further. No rule coerces an absent value to zero, and no rule computes a result from a value it did not read.

Three fields on the part specification carry a non-null default and none of them is a measured quantity. The name of the distributor a listing came from and the lifecycle status of a part both default to unknown, which records absence rather than substituting a value. The currency of a quoted price defaults to US dollars; this is the one assumed value in the specification and is recorded here as a known deviation from the paragraph above. No electrical, thermal, dimensional or stock field has a default.

The property shall be verified by test rather than by review. For each of the forty boards in the regression corpus, each field used by each rule is removed in turn, and the rule must report unchecked and name that field. A rule that produces a pass or a fail with the field removed is a defect.

8.5 Latency (DS-5)

A board of up to twelve slots produces a validated bill of materials within 130 seconds against live distributor data, and within 35 seconds against recorded data.

The budget is met in three ways. Slots are sourced concurrently rather than one after another, since sourcing is dominated by waiting on external calls. Numeric constraints are pushed into the distributor query so that the result set is small enough to normalise quickly, rather than fetched broadly and filtered afterwards. Repair is bounded, so the worst case is a known multiple of the normal case rather than an open-ended loop.

Latency is measured end to end from the receipt of the brief to the completion event, on boards of four, eight and twelve slots, against both live and recorded data. The recorded figure is the one used in continuous integration, because it is repeatable; the live figure is measured during integration testing and again during the demonstration period. The stages are timed separately so that a regression can be attributed to planning, sourcing, normalisation, validation or repair rather than to the run as a whole.

Two mechanisms carry the latency budget. Distributor lookups for the next likely candidates are started before they are needed, so that the network round trip overlaps with work already in progress. Normalised part specifications are cached, keyed by manufacturer part number and by a fingerprint of the instructions that produced the parse, so that a reworded instruction re-parses on its own. Stock, lifecycle and datasheet links are read live on every run and are never served from cache.

8.6 Streaming Responsiveness (DS-6)

The first event appears within two seconds of submission, and event sequences contain no gaps and no duplicates, including across a pause and a resume.

The two-second figure is met by emitting the acknowledgement of the run before any external call is made, so that the first event does not wait on a distributor or on the model. Ordering is met by giving each run a single writer: events are placed on one ordered queue for the run and the sequence number is assigned at that point, so that concurrent work inside the run cannot interleave its output. The high-water mark is persisted with the run rather than held in memory, which is what allows a resumed or continued run to carry on numbering correctly after the process that started it has gone.

The client tracks the last sequence number it rendered and discards anything at or below it. The test drives a run, kills the connection mid-stream, reconnects, and asserts that the union of what the client

received is the complete sequence from one to the completion event with no value missing and no value seen twice. The same assertion is made across an interrupt and its resume, and across an abandoned run and its continuation.

8.7 Bounded Repair (DS-7)

No slot is repaired more than three times before the conflict is raised to the engineer, and no run fails to terminate.

A counter is held per slot on the run state and is incremented when a repair is applied to that slot. At three, the engine stops offering repairs for that slot and emits an escalation describing the conflict as a trade-off. The counter lives on the run state rather than in the loop, so it survives a pause, a resume and a continuation from a checkpoint.

Termination follows from two bounds together. The number of slots on a board is bounded. The plan sets the initial count, and repair may add at most three further slots through the add_part action, after which no more may be added. Each slot, original or added, admits at most three repairs. The two bounds together give the number of repairs in a run a fixed ceiling. The property is verified against ten boards constructed to be unsatisfiable, each of which must terminate with an escalation rather than run on.

8.8 Sourceability (DS-9)

Every item in a bill of materials maps to a real manufacturer part number with sufficient distributor stock, or is clearly flagged where that cannot be met.

Sourcing pushes the numeric constraints of the slot into the distributor query rather than retrieving broadly and filtering afterwards, both to stay inside rate limits and to keep the set small enough to normalise within the latency budget. The results are normalised from free-text catalogue values into a single typed part specification, and every value keeps the field and source it came from.

A slot that cannot be filled with a real, sufficiently stocked part is reported as unresolved. It is never filled with a placeholder, a part whose stock is unknown, or a part assembled from values without an origin. Where a distributor source is unreachable, the affected slots are reported as unresolved rather than filled from a weaker source without saying so.

8.9 Model Fencing (DS-10)

Every repair the model proposes is checked against the engine’s set of allowed options before it is applied, and no unvalidated repair is accepted.

On a failure the engine computes the legal set: the participants in the conflict that may be changed, and for each the actions that are permitted. Six actions are defined — swap, change_topology, add_part, change_rail, relax_requirement and escalate. A slot the engineer named explicitly in the brief is restricted to swap, change_rail and escalate, so that a repair cannot quietly remove the function the engineer asked for. The set is ranked so that the cheapest change comes first.

The model selects one entry from that set. Its selection is checked for membership in the set before anything is applied. A selection outside the set, or one that does not arrive in time, is discarded and the

least disruptive candidate is used instead. The model is never asked for a number and has no route by which to emit one: it cannot produce a pass, a fail or a value, and it cannot mark a rule as checked.

This is also what contains prompt injection. A brief instructing the system to report that all checks have passed reaches a model that has no mechanism for emitting a pass. Validation stays with the engine rather than with a filter on the input. The property is evidenced by twenty-five injection briefs, each of which must produce zero false passes.

8.10 Data integrity

A run is recorded against a project. State that a run depends on is checkpointed as the run proceeds, so that a run which stops can continue from its last completed checkpoint rather than from an empty board. An operation that does not complete leaves no partial result that could be read as a finished one.

8.11 Security

Passwords are stored using argon2id. Session cookies are marked secure. No secret is committed to version control; secrets are provisioned through the deployment environment and are documented in the runbook.

8.12 Analytics Integrity

A rule that could not be evaluated is never counted as a pass and is never removed from a denominator to make a rate look cleaner. Every rate is reported with its numerator, its denominator and the number of evaluations that could not be completed.

Where the evaluations that could not be completed exceed twenty percent of a rule's denominator, no rate is reported for that rule. The counts are shown and the reason is named. A rate computed over a denominator that is mostly unknown is a substituted default in a different form.

No derived figure is written to storage. Persisting a score would place a board-dependent result in a record intended to hold facts intrinsic to a part, and would allow a stored figure to drift from the records it was computed from.

9 Output Requirements

9.1 Verdicts

The primary output is a verdict. Every verdict names the rule, the part it ran on, the outcome, the field that decided it, the value read from that field, and the source that value came from, as a resolvable link or a named reference. No verdict is emitted with an absent source. A rule that could not be evaluated is output as unchecked and names the field it needed.

9.2 The Validation Trace and the Component Graph

The trace shows each rule against each part with its outcome and its evidence. A conflict is shown together with every rule that ran on the failing part, including the rules that passed, so that an engineer

can decide whether the tool is right rather than only see that it objected. The component graph shows the slots, the placed parts and the relations used in validation.

9.3 The Bill of Materials

A completed run outputs a bill of materials in which every item maps to a real manufacturer part number with its price and stock, and carries the verdicts produced for it. Unresolved slots and unchecked rules appear as what they are.

9.4 Export

The bill of materials is exported as a CSV file containing the part number, quantity, price, stock and datasheet link for each item, reflecting the board after repair. The file opens correctly in Excel and in Google Sheets. Its column layout is an open issue.

9.5 Escalations

Where the system cannot safely decide how to proceed, the output is a question to the engineer rather than a decision. Where a conflict could not be repaired within the repair limit, the output is the conflict stated as an explicit trade-off. Neither is presented as a validated result.

9.6 Analysis

The analysis surface returns, for the authenticated engineer, the four views described in section 5.12. Each rate carries the counts it was derived from. Where a rate is withheld under section 6.12 the response carries the counts and the reason in place of the rate, and never a null presented as a zero.

10 Hardware Requirements

10.1 Network

An internet connection. The application is reached over HTTPS at a public address.

10.2 Client Computers

Any personal computer capable of running a current browser. No local installation is required.

10.3 Server

Hosted application and static site instances, and a managed PostgreSQL instance, on commercial hosting. A single application worker serves the event stream.

10.4 Bench Equipment

A fabricated test board and bench instruments are used during testing to compare the engine's thermal and current predictions against measured behaviour. This is measurement equipment for validation and is not part of the delivered system.

11 Software Requirements

11.1 Client

A current version of Chrome, Firefox, Edge or Safari, with JavaScript enabled.

11.2 Frontend

React 19, TypeScript, Vite and Tailwind CSS.

11.3 Application Server

Python 3.11 or later with FastAPI, serving JSON routes and one server-sent event stream.

11.4 Orchestration

LangGraph, which supplies the streaming, the pause and resume behaviour at an interrupt, and the persistent run state the workflow depends on.

11.5 Constraint Engine

Pure Python with no third-party runtime dependency, installable and runnable independently of the application server, with a documented programmatic interface.

11.6 Data Sources

The JLCPCB parts library for component parameters, stock and price. Mouser for lifecycle status and datasheet links. Manufacturer datasheets and web search as a fallback. Sources are reached through a Model Context Protocol tool layer so that one can be replaced without changing the engine.

11.7 Language Model

GLM-5.2, used to interpret the brief, to normalise free-text catalogue values, and to select a repair from the engine's legal set. It is used for nothing else.

11.8 Persistence

Managed PostgreSQL, holding accounts, projects, runs, workflow state and boards.

11.9 Testing

pytest and pytest-cov for unit, property and integration tests. Playwright for end-to-end browser tests.

11.10 Licenses

All components are open source or used within a free tier, except the language model inference credits, which are purchased. No licence obligation restricts delivery of the source to the sponsor.

12 Deployment Requirements

The application, the static site and the database are deployed to commercial hosting, reachable at a public HTTPS address. Environment variables and secrets are provisioned through the hosting environment and never committed. Database schema changes are applied as idempotent statements executed when the application starts. Each statement is written so that running it against a schema that already contains the change has no effect, which makes deployment and restart safe to repeat and removes the need to track which changes have been applied.

Cold start completes within 60 seconds and a warm health check responds within 500 milliseconds. Availability of at least 99 percent is maintained across the two-week demonstration period. Monitoring covers availability, error rate and run completion.

A runbook documents the environment variables, secret provisioning, the schema-change procedure, the corrective procedure for reverting a schema change, monitoring, and a local-hosting fallback that allows the system to be demonstrated without the hosted environments.